

EXPERIMENT -2

Student Name: Vinay Tiwari

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70919

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

Question 1)

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
Select d.dept_name as Department,
round(avg(m.marks_scored), 2) as "Average Marks"
from Departments as d
join students as s
on d.dept_id =s.dept_id
join marks as m
on m.student_id=s.student_id
group by d.dept_name
order by "Average Marks" Desc
```

EXPERIMENT -2

OUTPUT SCREENSHOTS:

	department character varying (50) 	Average Marks numeric 
1	Mathematics	95.00
2	Computer Science	83.33

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.
- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

CONCLUSION:

The PostgreSQL query successfully generated a department-wise performance report by joining the relevant tables, calculating the average marks using the **AVG()** function, grouping the data by department, and sorting the results in descending order. This experiment demonstrated the effective use of SQL joins, aggregate functions, grouping, and sorting to perform data analysis and produce useful academic performance reports.

EXPERIMENT -2

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
select C.empid, min(c.ename), min(c.salary) from
(select * from A
union all
select * from B) as C
group by c.empid
```

EXPERIMENT -2

OUTPUT SCREENSHOTS:

	empid integer	min text	min integer
1	3	CC	100
2	2	BB	300
3	1	AA	1000

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.
- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

CONCLUSION:

The PostgreSQL query successfully generated a department-wise performance report by joining the relevant tables, calculating the average marks using the **AVG()** function, grouping the data by department, and sorting the results in descending order. This experiment demonstrated the effective use of SQL joins, aggregate functions, grouping, and sorting to perform data analysis and produce useful academic performance reports.

EXPERIMENT -2

A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name**, **Employee Name**, and **Salary**, arranged in ascending order of department name.

Note: The employee and department information are maintained in separate normalized tables.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
select d.dept_name, e.name, e.salary from Employee as e
join department as d
on d.id=e.department_id
where e.salary = (select max(e2.salary) from employee as e2
where e2.department_id=d.id)
```

OUTPUT SCREENSHOTS:

	dept_name character varying (50) 🔒	name character varying (50) 🔒	salary integer 🔒
1	IT	JIM	90000
2	SALES	HENRY	80000
3	IT	MAX	90000

EXPERIMENT -2

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to retrieve data from multiple normalized tables using JOIN operations.
- Learn to identify employees with the highest salary within each department using SQL queries.
- Apply aggregate functions such as MAX() to determine the maximum salary in each department.
- Use subqueries or equivalent SQL techniques to compare employee salaries with the highest departmental salary.
- Handle situations where multiple employees share the highest salary, ensuring all qualifying employees are included in the result.

CONCLUSION:

The SQL query successfully identified the employee(s) with the highest salary in each department by joining the employee and department tables and using the MAX() aggregate function with a subquery. It correctly included all employees who shared the highest salary within their respective departments and displayed the department name, employee name, and salary in ascending order of department name. This experiment demonstrated the effective use of SQL joins, aggregate functions, subqueries, and sorting techniques to generate meaningful organizational reports.